Informe Tecnico — GroupsApp

Curso: ST0263 Topicos Especiales en Telematica / SI3007 Sistemas Distribuidos
Semestre: 2026-1
Integrantes: Camilo Sanchez Salinas, Samuel Santamaria, Yessetk Rodriguez

---

1. Descripcion General

GroupsApp es una aplicacion de mensajeria instantanea con arquitectura de microservicios desplegada en AWS. Permite comunicacion en tiempo real mediante grupos con canales multiples y mensajes directos entre usuarios, con soporte para envio de archivos e imagenes.

---

2. Arquitectura del Sistema

2.1 Diagrama de Componentes

```
                    Internet
                        |
             AWS Elastic Load Balancer
                        |
           +-----------+-----------+
           |                       |
     [frontend :80]          [backend :3000]
      React + Nginx           Express + Socket.IO
                                   |
             +---------------------+---------------------+
             |                     |                     |
        [RabbitMQ]             [Redis]             [RDS MySQL]
         AMQP broker           cache/presencia      datos principales
             |
      +-------+-------+
      |               |
[messages-service] [groups-service]
   :50051 gRPC        :50052 gRPC
   AMQP consumer      AMQP consumer
                            |
                         [S3]
                     archivos/imagenes
```

2.2 Descripcion de Servicios

Servicio	Tecnologia	Puerto	Responsabilidad
`frontend`	React 18, Vite, Nginx, TailwindCSS	80	Interfaz de usuario SPA
`backend`	Node.js, Express, Socket.IO	3000	API REST, WebSocket gateway, AMQP publisher

`messages-service`	Node.js, gRPC, amqplib	50051	Persistencia mensajes, AMQP consumer
`groups-service`	Node.js, gRPC, amqplib	50052	Gestion grupos/canales/miembros
`rabbitmq`	RabbitMQ 3.13	5672 / 15672	Broker de eventos asincronos
`redis`	Redis 7	6379	Cache presencia online, sesiones

2.3 Patrones de Comunicacion

El sistema implementa tres tipos de comunicacion:

Comunicacion sincrona REST (cliente <-> backend)
El frontend consume la API REST del backend para todas las operaciones CRUD. Autenticacion mediante JWT en header `Authorization: Bearer <token>`.

Comunicacion sincrona gRPC (backend <-> microservicios)
El backend actua como cliente gRPC hacia messages-service y groups-service para consultas internas. Los protobuf estan definidos en `proto/messages.proto` y `proto/groups.proto`.

Comunicacion asincrona AMQP (backend -> RabbitMQ -> microservicios)
Al recibir un mensaje via Socket.IO, el backend publica un evento `message.sent` en el exchange `groupsapp.events`. Messages-service consume y persiste en MySQL. Patron publish/subscribe con topic exchange.

WebSocket (cliente <-> backend)
Socket.IO gestiona la mensajeria en tiempo real, salas por grupo/canal y presencia online/offline.

2.4 Infraestructura AWS

Recurso	Configuracion	Proposito
EKS	Cluster K8s, 2x t3.medium, auto-scaling 2-4 nodos	Orquestacion contenedores
RDS MySQL	db.t3.micro, Single-AZ, MySQL 8.0	Base de datos relacional
S3	Bucket `groupsapp-files-<account-id>`, us-east-1	Archivos e imagenes
ECR	4 repositorios (backend, frontend, messages, groups)	Registry de imagenes Docker
ELB	LoadBalancer por servicio via K8s type:LoadBalancer	Acceso publico con HA

2.5 Escalabilidad y Alta Disponibilidad

- **HPA (Horizontal Pod Autoscaler)**: backend, messages-service y groups-service escalan entre 1-2 replicas segun CPU/memoria (umbral 80%)
- **Backend con 2 replicas fijas** para demo de load balancing
- **ELB distribuye trafico** entre pods en nodos distintos
- **RDS Single-AZ** (limitacion de AWS Academy, sin Multi-AZ)

---

3. Modelo de Datos

3.1 Diagrama Entidad-Relacion (simplificado)

```
users (1) ----< group_members >---- (N) groups
users (1) ----< group_messages (group_id, channel_id)
users (1) ----< messages (sender_id, receiver_id) — DMs
groups (1) ----< channels
groups (1) ----< group_messages
contacts: user_id <-> contact_user_id
message_reads: message_id + user_id
```

3.2 Esquema SQL Completo

```sql
CREATE TABLE users (
    id           VARCHAR(36)  PRIMARY KEY,
    name         VARCHAR(100) NOT NULL,
    username     VARCHAR(50)  UNIQUE,
    email        VARCHAR(150) NOT NULL UNIQUE,
    password     VARCHAR(255) NOT NULL,
    profile_picture VARCHAR(500),
    is_online    BOOLEAN      DEFAULT FALSE,
    last_seen    DATETIME,
    created_at   DATETIME     DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE groups (
    id          VARCHAR(36)  PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    description TEXT,
    created_by  VARCHAR(36)  NOT NULL REFERENCES users(id),
    join_mode   ENUM('open','request','invite') DEFAULT 'open',
    is_public   BOOLEAN      DEFAULT TRUE,
    created_at  DATETIME     DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE group_members (
    id        INT AUTO_INCREMENT PRIMARY KEY,
    group_id  VARCHAR(36) NOT NULL REFERENCES groups(id) ON DELETE CASCADE,
    user_id   VARCHAR(36) NOT NULL REFERENCES users(id)  ON DELETE CASCADE,
    role      ENUM('admin','member') DEFAULT 'member',
    joined_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (group_id, user_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE channels (
    id          VARCHAR(36)  PRIMARY KEY,
    group_id    VARCHAR(36)  NOT NULL REFERENCES groups(id) ON DELETE CASCADE,
    name        VARCHAR(100) NOT NULL,
    description TEXT,
    created_by  VARCHAR(36)  NOT NULL REFERENCES users(id),
    created_at  DATETIME     DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE group_messages (
```

```sql
    id           VARCHAR(36) PRIMARY KEY,
    group_id     VARCHAR(36) NOT NULL REFERENCES groups(id) ON DELETE CASCADE,
    channel_id   VARCHAR(36),
    sender_id    VARCHAR(36) NOT NULL REFERENCES users(id),
    content      TEXT,
    file_url     VARCHAR(500),
    file_name    VARCHAR(255),
    file_type    VARCHAR(100),
    message_type ENUM('text','file','image') DEFAULT 'text',
    status       ENUM('sent','delivered','read') DEFAULT 'sent',
    created_at   DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE messages (
    id           VARCHAR(36) PRIMARY KEY,
    sender_id    VARCHAR(36) NOT NULL REFERENCES users(id),
    receiver_id  VARCHAR(36) NOT NULL REFERENCES users(id),
    content      TEXT,
    file_url     VARCHAR(500),
    file_name    VARCHAR(255),
    file_type    VARCHAR(100),
    message_type ENUM('text','file','image') DEFAULT 'text',
    status       ENUM('sent','delivered','read') DEFAULT 'sent',
    read         BOOLEAN DEFAULT FALSE,
    created_at   DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE contacts (
    id              VARCHAR(36) PRIMARY KEY,
    user_id         VARCHAR(36) NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    contact_user_id VARCHAR(36) NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    created_at      DATETIME DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (user_id, contact_user_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE message_reads (
    id         VARCHAR(36) PRIMARY KEY,
    message_id VARCHAR(36) NOT NULL,
    user_id    VARCHAR(36) NOT NULL,
    read_at    DATETIME DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (message_id, user_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

3.3 Decisiones de Diseno

- **IDs en UUID v4 (VARCHAR 36)**: evita colisiones en entorno distribuido, facilita migracion entre instancias
- **utf8mb4**: soporte completo Unicode incluyendo emojis
- **ON DELETE CASCADE**: integridad referencial automatica al eliminar grupos/miembros
- **Separacion group_messages / messages**: permite consultas optimizadas por tipo sin JOINs costosos
- **Redis para presencia**: estado volatile, no necesita durabilidad ACID; latencia sub-milisegundo

---

4. API REST

Base URL: `http://<backend-lb>:3000`
Autenticacion: Header `Authorization: Bearer <JWT>`
Formato respuesta: `{ "success": boolean, "data": any }` o `{ "success": false, "error": string }`

4.1 Autenticacion

POST /api/auth/register

Registra nuevo usuario.

Request:

```
{
  "name": "Juan Perez",
  "email": "juan@example.com",
  "password": "minimo8chars"
}
```

Response 201:

```
{
  "success": true,
  "data": {
    "token": "eyJhbGci...",
    "user": { "id": "uuid", "name": "Juan Perez", "email": "juan@example.com" }
  }
}
```

POST /api/auth/login

Autentica usuario, retorna JWT.

Request:

```
{ "email": "juan@example.com", "password": "minimo8chars" }
```

Response 200:

```
{ "success": true, "data": { "token": "eyJhbGci...", "user": { ... } } }
```

GET /api/auth/me

Retorna perfil del usuario autenticado.

Response 200:

```
{ "success": true, "data": { "id": "uuid", "name": "...", "email": "...", "is_online": true
} }
```

4.2 Grupos

GET /api/groups

Lista grupos del usuario autenticado (donde es miembro).

Response 200:

```
{
  "success": true,
  "data": [
    { "id": "uuid", "name": "Backend Team", "description": "...", "role": "admin",
"member_count": 5 }
  ]
}
```

POST /api/groups

Crea grupo. El creador se agrega automaticamente como admin.

Request:

```
{ "name": "Backend Team", "description": "Opcional", "is_public": true }
```

GET /api/groups/:id

Detalles de grupo incluyendo lista de miembros.

DELETE /api/groups/:id

Elimina grupo y todos sus mensajes/canales. Solo admin.

POST /api/groups/:id/members

Agrega miembro al grupo.

Request: `{ "userId": "uuid" }`

DELETE /api/groups/:id/members/:userId

Remueve miembro. Solo admin.

4.3 Canales

GET /api/groups/:id/channels

Lista canales del grupo.

POST /api/groups/:id/channels

Crea canal en el grupo. Solo miembros.

Request: `{ "name": "general", "description": "Opcional" }`

DELETE /api/groups/:id/channels/:channelId

Elimina canal. Solo admin del grupo.

4.4 Mensajes

GET /api/messages/group/:id?channelId=uuid&limit=50&offset=0

Historial de mensajes de grupo o canal especifico.

Response 200:

```
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "content": "Hola mundo",
      "message_type": "text",
      "sender": { "id": "uuid", "name": "Juan", "profile_picture": null },
      "created_at": "2026-04-26T14:00:00Z"
    }
  ]
}
```

GET /api/messages/direct/:userId?limit=50&offset=0

Historial DM con un usuario.

DELETE /api/messages/:id

Elimina mensaje propio. Solo el sender puede eliminar.

4.5 Contactos

GET /api/contacts

Lista contactos del usuario.

POST /api/contacts

Agrega contacto.

Request: `{ "contactUserId": "uuid" }` o `{ "email": "contacto@example.com" }`

DELETE /api/contacts/:id

Elimina contacto.

4.6 Archivos

POST /api/files/upload

Sube archivo a S3. Multipart/form-data.

Campo: `file` (max 10MB).

Response 200:

```
{ "success": true, "data": { "url": "https://s3.amazonaws.com/...", "key":
```

```
"uploads/uuid.jpg" } }
```

GET /api/files/signed-url/:key

Genera URL firmada de S3 valida por 1 hora.

4.7 Sistema

GET /api/health

```
{ "status": "ok", "timestamp": "2026-04-26T14:00:00Z" }
```

GET /metrics

Metricas en formato Prometheus (counter, gauge, histogram para requests HTTP).

---

5. WebSocket

Libreria: Socket.IO 4.x
Autenticacion: Token JWT en handshake `socket.handshake.auth.token`

5.1 Eventos Cliente -> Servidor

Evento	Payload	Descripcion
`join_group`	`{ groupId: string }`	Suscribirse a sala del grupo
`join_channel`	`{ channelId: string }`	Suscribirse a sala de canal
`send_message`	`{ groupId, channelId, content, type }`	Enviar mensaje a grupo/canal
`send_direct`	`{ receiverId, content, type }`	Enviar mensaje directo
`typing`	`{ groupId, channelId }`	Notificar que usuario esta escribiendo

5.2 Eventos Servidor -> Cliente

Evento	Payload	Descripcion
`new_message`	MessageObject	Nuevo mensaje en grupo/canal
`new_direct_message`	MessageObject	Nuevo DM recibido
`message_deleted`	`{ messageId }`	Mensaje eliminado
`user_online`	`{ userId }`	Usuario se conecta
`user_offline`	`{ userId }`	Usuario se desconecta
`typing`	`{ userId, groupId, channelId }`	Indicador de escritura

5.3 Gestion de Presencia

Al conectar: `SET user:<id>:online 1 EX 86400` en Redis + emit `user_online` a contactos.
Al desconectar: DEL key + UPDATE `last_seen` en MySQL + emit `user_offline`.

---

6. gRPC

6.1 MessagesService (proto/messages.proto)

```
service MessagesService {
  rpc SaveMessage(MessageRequest) returns (MessageResponse);
  rpc GetMessages(GetMessagesRequest) returns (MessagesResponse);
  rpc GetDirectMessages(GetDirectRequest) returns (MessagesResponse);
}
```

6.2 GroupsService (proto/groups.proto)

```
service GroupsService {
  rpc GetGroup(GroupRequest) returns (GroupResponse);
  rpc GetMembers(GroupRequest) returns (MembersResponse);
  rpc AddMember(AddMemberRequest) returns (MemberResponse);
  rpc GetChannels(GroupRequest) returns (ChannelsResponse);
}
```

---

7. RabbitMQ

Configuracion	Valor
Exchange	`groupsapp.events` (type: `topic`)
Routing key	`message.sent`
Queue (messages-service)	`messages.persist`
Durabilidad	Exchange y queue durables, mensajes persistentes

Flujo:

1. Backend recibe mensaje via Socket.IO
2. Backend emite `new_message` a sala de Socket.IO (tiempo real)
3. Backend publica `message.sent` en exchange (asincrono)
4. messages-service consume y persiste en MySQL
5. Estado de mensaje actualizado a `delivered`

---

8. Seguridad

Mecanismo	Implementacion
Autenticacion	JWT (jsonwebtoken), expira en 7 dias
Hash contrasenas	bcrypt, cost factor 10

Headers HTTP	Helmet.js (X-Frame-Options, CSP, HSTS, etc.)
Rate limiting	express-rate-limit en `/api/auth/*`: 10 req/15min por IP
CORS	Lista blanca en variable de entorno `CORS_ORIGINS`
Upload	multer limita a 10MB, tipos MIME validados
Credenciales AWS	IMDS en EC2/EKS, variables de entorno en desarrollo

---

9. Observabilidad

9.1 Logs

Winston con niveles `error`, `warn`, `info`, `debug`. Formato JSON en produccion. Los servicios escriben a stdout (capturado por Kubernetes) y a `/logs/*.log`.

9.2 Metricas Prometheus

Endpoint `/metrics` en backend (:3000), messages-service (:9091) y groups-service (:9092).

Metricas expuestas:

- `http_requests_total` (counter, por ruta y status)
- `http_request_duration_seconds` (histogram)
- `active_websocket_connections` (gauge)
- `rabbitmq_messages_published_total` (counter)
- `rabbitmq_messages_consumed_total` (counter)

---

10. Pruebas

10.1 Estructura

```
backend/__tests__/
    routes/
        auth.test.js       -- register, login, me, token invalido
        groups.test.js     -- CRUD grupos, roles, permisos
        channels.test.js   -- CRUD canales
        messages.test.js   -- historial grupal y DM, eliminacion
        contacts.test.js   -- agregar, listar, eliminar
        files.test.js      -- upload S3, signed URL

messages-service/__tests__/
    handlers/              -- SaveMessage, GetMessages

groups-service/__tests__/
    handlers/              -- GetGroup, GetMembers, AddMember
```

10.2 Herramientas

- **Jest** — test runner
- **Supertest** — HTTP assertions sobre Express

- **Mock** de MySQL pool para tests unitarios de rutas
- **Mock** de S3Client para tests de upload

10.3 Casos de Prueba Principales

Autenticacion

Caso	Resultado esperado
Register con datos validos	201, retorna token y usuario
Register con email duplicado	409, `error: "Email already registered"`
Login con credenciales correctas	200, retorna token
Login con password incorrecta	401, `error: "Invalid credentials"`
GET /me con token valido	200, retorna perfil
GET /me sin token	401, `error: "No token provided"`
GET /me con token expirado	401, `error: "Token expired"`

Grupos

Caso	Resultado esperado
Crear grupo autenticado	201, creador es admin
Listar grupos del usuario	200, solo grupos donde es miembro
Eliminar grupo como admin	200, cascade elimina mensajes y canales
Eliminar grupo como member	403, `error: "Admin required"`
Agregar miembro existente	201
Agregar miembro ya en grupo	409

Mensajes

Caso	Resultado esperado
Obtener historial de grupo	200, array de mensajes con sender
Eliminar mensaje propio	200
Eliminar mensaje ajeno	403

Archivos

Caso	Resultado esperado
Upload archivo valido	200, retorna URL S3
Upload sin archivo	400

Signed URL con key valida	200, retorna URL temporal

10.4 Ejecucion

```
# Backend (desde /backend)
npm test

# Con coverage
npm test -- --coverage

# Servicio especifico
npm test -- --testPathPattern=auth
```

10.5 Resultados

Los tests corren con mocks de DB y S3, no requieren infraestructura activa. Cobertura aproximada: 85% rutas backend, 70% handlers gRPC.

---

11. Despliegue

11.1 Kubernetes (EKS)

Los manifests en `k8s/` definen:

Archivo	Recursos
`namespace.yaml`	Namespace `groupsapp`
`configmap.yaml`	Variables de entorno no sensibles
`secret.yaml`	Credenciales DB, JWT, AWS
`redis.yaml`	Deployment + Service ClusterIP
`rabbitmq.yaml`	Deployment + Service LoadBalancer
`backend.yaml`	Deployment (2 replicas) + Service LoadBalancer + HPA
`frontend.yaml`	Deployment + Service LoadBalancer + HPA
`ingress.yaml`	Ingress (opcional, usando LB directo)

11.2 Docker Compose (local)

`docker-compose.yml` levanta todos los servicios en red interna `groupsapp-network`. Incluye volumen para datos MySQL local.

11.3 CI/CD (manual)

```
# Build y push imagen
docker build -t groupsapp-backend ./backend
```

```
docker tag groupsapp-backend:latest <ecr>/<repo>:latest
docker push <ecr>/<repo>:latest

# Rollout
kubectl rollout restart deployment/backend -n groupsapp
```

---

12. Limitaciones y Trabajo Futuro

Limitacion	Causa	Solucion potencial
RDS Single-AZ	Restriccion AWS Academy	RDS Multi-AZ en produccion
Sin TLS en LB	Certificado requiere dominio propio	ACM + Route 53
Presencia no persistente ante crash	Redis volatile	Redis con AOF + persistencia
Sin etcd activo	Simplificacion implementacion	Agregar etcd para service discovery
Imagenes Docker sin tag versionado	Solo `latest`	Tags semanticos + GitOps